



UNIVERSIDADE ESTADUAL DA PARAÍBA
CIÊNCIAS DA COMPUTAÇÃO

DAVID EMMANUEL XAVIER DOS SANTOS

GABRIEL TRIGUEIRO FERNANDES

**PROJETO EXPERIMENTAL: ANÁLISE COMPARATIVA DE TÉCNICAS DE PROJETO
DE ALGORITMOS**

CAMPINA GRANDE

2026

1. Introducao

O Problema do Subconjunto Soma (Subset Sum) é um problema clássico da teoria da computação: dado um conjunto de n inteiros positivos e um valor alvo T , determinar se existe algum subconjunto cujos elementos somam exatamente T .

Formato de entrada:

```
n T
a1 a2 a3 ... an
```

Formato de saída (com solução):

```
YES
k
i1 i2 ... ik
```

Formato de saída (sem solução):

```
NO
```

O Subset Sum é um problema NP-completo. Isso significa que, até hoje, não se conhece algoritmo capaz de resolvê-lo em tempo polinomial no caso geral. No pior caso, é necessário examinar todos os 2^n subconjuntos possíveis. Por esse motivo, técnicas de exploração inteligente do espaço de busca - como poda, limites e memorização - são fundamentais para tornar o problema tratável na prática.

Este relatório descreve, analisa e compara duas dessas técnicas:

- Backtracking - busca em profundidade com poda
- Branch and Bound - busca em largura com limite superior

2. Algoritmos Implementados

2.1 Backtracking

Descricao

O Backtracking percorre uma árvore binária de decisão em profundidade (DFS - Depth-First Search). Em cada nível i da árvore, o algoritmo decide se o elemento $nums[i]$ será incluído ou excluído do subconjunto atual. Ao final de cada caminho, ou a soma acumulada atingiu o alvo - solução encontrada - ou esgotaram-se os elementos sem atingi-lo.

A grande vantagem do DFS é que ele abandona imediatamente qualquer ramo que se mostre inviável, voltando (backtracking) para o nó anterior e tentando o outro caminho.

Estratégia de Poda

Duas condições disparam uma poda:

- Poda por excesso: $\text{nums}[\text{index}] > \text{remaining}$ - incluir o elemento ultrapassaria o alvo restante; o ramo da inclusão é descartado.
- Poda por insuficiência: $\text{sum}(\text{nums}[\text{index}:]) < \text{remaining}$ - mesmo somando todos os elementos restantes, não é possível atingir o alvo; o ramo inteiro é descartado.

Pseudocódigo

```
funcao backtrack(index, remaining, current):
    se remaining == 0:
        SOLUCAO ENCONTRADA - retorna current
    se index == n:
        retorna
    se sum(nums[index:]) < remaining:    <- poda por insuficiencia
        retorna
    se nums[index] <= remaining:         <- ramo INCLUIR
        current.adiciona(index)
        backtrack(index+1, remaining - nums[index], current)
        current.remove(index)          <- backtrack
    senao:
        PODA (poda por excesso)
        backtrack(index+1, remaining, current) <- ramo EXCLUIR
```

Complexidade

Aspecto	Valor
Pior caso (sem poda eficaz)	$O(2^n)$
Melhor caso (solução imediata)	$O(n)$
Espaco (pilha de recursão)	$O(n)$
Estrutura	DFS recursivo

2.2 Branch and Bound

Descricao

O Branch and Bound explora a árvore de decisão em largura (BFS - Breadth-First Search) usando uma fila explícita de estados. Cada estado representa uma decisão parcial: quais elementos foram incluídos até o índice atual e qual é a soma acumulada.

A diferença fundamental em relação ao Backtracking é o cálculo de um limite superior antes de expandir qualquer estado. Se esse limite mostrar que o alvo é inalcançável, o estado é descartado sem gerar filhos.

Para acelerar o cálculo, o algoritmo pre-computa as somas de sufixo do array:

```
suffix_sum[i] = nums[i] + nums[i+1] + ... + nums[n-1]
```

Assim, o limite superior de qualquer estado é calculado em $O(1)$:

```
limite_superior = current_sum + suffix_sum[index]
```

Estratégia de Poda

Tres condicoes de poda são verificadas:

- Poda por excesso (ramo inclusão): $\text{current_sum} + \text{nums}[\text{index}] > \text{target}$.
- Poda por insuficiência: $\text{limite_superior} < \text{target}$ - mesmo somando tudo que resta, não se atinge o alvo.
- Poda do ramo exclusão: $\text{suffix_sum}[\text{index}+1] < \text{target} - \text{current_sum}$ - sem o elemento atual, os restantes não bastam.

Pseudocodigo

```
pre-computa suffix_sum[0..n]

fila <- [(index=0, current_sum=0, selecionados=[])]

enquanto fila nao vazia:
    (index, current_sum, selecionados) <- fila.dequeue()
    se current_sum == target:
        SOLUCAO ENCONTRADA - retorna selecionados
    se index == n: continua
    upper_bound <- current_sum + suffix_sum[index]
    se upper_bound < target:                <- poda insuficiencia
        continua
    se current_sum + nums[index] <= target:  <- ramo INCLUIR
        fila.enqueue((index+1, current_sum + nums[index], ...))
    senao: PODA (excesso)
    se suffix_sum[index+1] >= target - current_sum: <- ramo EXCLUIR
        fila.enqueue((index+1, current_sum, selecionados))
    senao: PODA (insuficiencia sem elemento)
```

Complexidade

Aspecto	Valor
Pior caso (sem poda eficaz)	$O(2^n)$
Pre-computo suffix_sum	$O(n)$
Espaco (fila de estados)	$O(2^n)$ pior caso
Estrutura	BFS com fila explicita

3. Cenários de Teste

Os testes foram realizados em sete cenários distintos, cada um projetado para evidenciar características específicas dos algoritmos:

#	Arquivo	n	T	Conjunto	Característica
1	input.txt	4	9	{3, 4, 5, 8}	Caso base, pequeno
2	padrao.txt	8	15	{3, 5, 7, 2, 8, 1, 4, 6}	Caso equilibrado - base de comparação
3	solucao_imediata.txt	10	12	{5, 7, 9, 3, 8, 2, 6, 4, 1, 10}	Solução nos 2 primeiros elementos
4	sem_solucao.txt	12	79	{1..12}	Sem solução (soma total = $78 < 79$)
5	poda_maxima.txt	10	5	{20, 18, 15, ..., 2}	Elementos grandes, alvo pequeno - máxima poda
6	solucao_no_final.txt	10	9	{4x9, 9}	Solucao apenas no ultimo elemento
7	fila_grande_bb.txt	15	30	{2, 3, 4, ..., 16}	Muitos caminhos viáveis - pior caso do B&B

4. Resultados Experimentais

4.1 Dados por Cenário

Cenário 1 - input.txt (n=4, T=9)

Métrica	Backtracking	Branch and Bound
Solucao encontrada	SIM - {4, 5}	SIM - {4, 5}
Nos visitados	13	11
Podas realizadas	4	8
Eficiencia de poda	23,5%	42,1%
Prof. max. / Fila max.	4	6
Tempo de execucao	0,0520 ms	0,0596 ms
Pico de memoria	0,36 KB	1,55 KB

Observação: Caso pequeno. B&B elimina mais ramos (42,1% de poda vs 23,5%), porém ambos são rápidos e consomem pouca memória.

Cenário 2 - padrao.txt (n=8, T=15) - Caso Base

Metrica	Backtracking	Branch and Bound
Solucao encontrada	SIM - {3, 5, 7}	SIM - {3, 5, 7}
Nos visitados	4	8
Podas realizadas	0	0
Eficiencia de poda	0,0%	0,0%
Prof. max. / Fila max.	3	8
Tempo de execucao	0,0105 ms	0,0255 ms
Pico de memoria	0,28 KB	1,12 KB

Observação: O Backtracking encontrou a solução {3, 5, 7} no terceiro nível da árvore DFS, visitando apenas 4 nós. O B&B percorreu 8 estados via BFS antes de encontrar a mesma solução - a exploração por largura não favorece casos com solução rasa e precoce.

Cenário 3 - solucao_imediata.txt (n=10, T=12) - Vantagem do Backtracking

Metrica	Backtracking	Branch and Bound
Solucao encontrada	SIM - {5, 7}	SIM - {5, 7}
Nos visitados	3	4
Podas realizadas	0	0
Eficiencia de poda	0,0%	0,0%
Prof. max. / Fila max.	2	4
Tempo de execucao	0,0085 ms	0,0101 ms
Pico de memoria	0,38 KB	0,78 KB

Observação: A solução está nos dois primeiros elementos. O DFS do Backtracking desce direto para a resposta em 2 decisões. O BFS do B&B é quase tão eficiente (4 estados), mas usa mais memória por manter a fila.

Cenário 4 - sem_solucao.txt (n=12, T=79) - Sem Solucao

Metrica	Backtracking	Branch and Bound
Solucao encontrada	NAO	NAO
Nos visitados	1	1
Podas realizadas	1	1
Eficiencia de poda	50,0%	50,0%
Prof. max. / Fila max.	0	1
Tempo de execucao	0,0046 ms	0,0026 ms
Pico de memoria	0,37 KB	0,74 KB

Observação: A soma total do conjunto é 78 (1+2+...+12), menor que o alvo 79. Ambos detectam a impossibilidade imediatamente no primeiro nó, via poda por insuficiência (suffix_sum inteiro < target).

Cenário 5 - poda_maxima.txt (n=10, T=5) - Maxima Poda

Métrica	Backtracking	Branch and Bound
Solucao encontrada	SIM - {3, 2}	SIM - {3, 2}
Nos visitados	14	13
Podas realizadas	9	12
Eficiencia de poda	39,1%	48,0%
Prof. max. / Fila max.	10	2
Tempo de execucao	0,0287 ms	0,0147 ms
Pico de memoria	0,35 KB	0,83 KB

Observação: Elementos grandes (20, 18, 15...) comparados ao alvo pequeno (5). O B&B tem eficiência de poda superior (48% vs 39%) e é mais rápido (0,015 ms vs 0,029 ms). A fila máxima foi apenas 2 estados - a poda agressiva manteve o BFS compacto.

Cenário 6 - solucao_no_final.txt (n=10, T=9) - Ponto Fraco do Backtracking

Métrica	Backtracking	Branch and Bound
Solucao encontrada	SIM - {9}	SIM - {9}
Nos visitados	221	176
Podas realizadas	129	175
Eficiencia de poda	36,9%	49,9%
Prof. max. / Fila max.	10	46
Tempo de execucao	0,3459 ms	0,2902 ms
Pico de memoria	0,38 KB	6,73 KB

Observação: O conjunto {4,4,4,4,4,4,4,4,9} tem a solução no último elemento. O Backtracking explora profundamente todos os ramos com os 4s antes de chegar ao 9. O B&B visita menos nos (176 vs 221) e é mais rápido, mas paga com memória: fila chegou a 46 estados, consumindo 6,73 KB contra 0,38 KB do Backtracking.

Cenário 7 - fila_grande_bb.txt (n=15, T=30) - Ponto Fraco do Branch and Bound

Métrica	Backtracking	Branch and Bound
Solucao encontrada	SIM - {2,3,4,5,6,10}	SIM - {2,3,4,6,7,8}
Nos visitados	37	135
Podas realizadas	24	8
Eficiencia de poda	39,3%	5,6%

Prof. max. / Fila max.	15	127
Tempo de execucao	0,0640 ms	0,5685 ms
Pico de memoria	0,41 KB	14,10 KB

Observação: Com 15 elementos consecutivos e alvo 30, há muitas combinações que satisfazem a poda, impedindo o B&B de descartar estados cedo. A fila cresce até 127 estados simultâneos. O Backtracking encontra solução no 37 no e para imediatamente: 8,9x mais rápido e 34x menos memória.

4.2 Tabela Consolidada

Cenario	Alg.	Nos Visit.	Podas	Efic. Poda	Tempo (ms)	Mem. (KB)	Solucao
input (n=4)	BT	13	4	23,5%	0,0520	0,36	SIM
input (n=4)	B&B	11	8	42,1%	0,0596	1,55	SIM
padrao (n=8)	BT	4	0	0,0%	0,0105	0,28	SIM
padrao (n=8)	B&B	8	0	0,0%	0,0255	1,12	SIM
sol_imediata (n=10)	BT	3	0	0,0%	0,0085	0,38	SIM
sol_imediata (n=10)	B&B	4	0	0,0%	0,0101	0,78	SIM
sem_solucao (n=12)	BT	1	1	50,0%	0,0046	0,37	NAO
sem_solucao (n=12)	B&B	1	1	50,0%	0,0026	0,74	NAO
poda_maxima (n=10)	BT	14	9	39,1%	0,0287	0,35	SIM
poda_maxima (n=10)	B&B	13	12	48,0%	0,0147	0,83	SIM
sol_no_final (n=10)	BT	221	129	36,9%	0,3459	0,38	SIM
sol_no_final (n=10)	B&B	176	175	49,9%	0,2902	6,73	SIM

fila_grande (n=15)	BT	37	24	39,3%	0,0640	0,41	SIM
fila_grande (n=15)	B&B	135	8	5,6%	0,5685	14,10	SIM

5. Análise Comparativa

5.1 Complexidade Assintótica (Teórica)

Ambos os algoritmos possuem complexidade $O(2^n)$ no pior caso. A diferença está no comportamento médio e no uso de memória:

Critério	Backtracking	Branch and Bound
Complexidade de tempo	$O(2^n)$	$O(2^n)$
Complexidade de espaço	$O(n)$ - pilha de recursão	$O(2^n)$ - fila de estados
Estratégia de busca	DFS (profundidade)	BFS (largura)
Overhead por estado	Baixo (sem alocação extra)	Alto (copia da lista de selecionados)

O B&B tem espaço potencialmente exponencial porque a fila pode armazenar simultaneamente todos os estados de um nível da árvore. O Backtracking usa apenas a pilha de recursão, limitada a profundidade n .

5.2 Tempo de Execução (Empírico)

- O Backtracking foi mais rápido em 5 dos 7 cenários.
- O B&B foi mais rápido nos cenários de poda intensa e sem solução (cenários 4 e 5).
- A maior diferença: cenário fila_grande_bb (n=15) - BT foi 8,9x mais rápido (0,064 ms vs 0,569 ms).
- No cenário sol_no_final, o B&B foi levemente mais rápido (0,29 ms vs 0,35 ms), ao custo de 17x mais memória.

5.3 Uso de Memória

O Backtracking apresenta consumo de memória praticamente constante e baixo em todos os cenários (0,28 a 0,41 KB), pois usa apenas a pilha de recursos com profundidade limitada a n .

O B&B tem comportamento de memória diretamente dependente do número de caminhos viáveis simultâneos:

- Cenários com poda eficaz -> fila pequena -> memória baixa (~0,74 a 1,55 KB)

- Cenário fila_grande_bb -> fila de 127 estados -> 14,10 KB (34x mais que o Backtracking)
- Cenário sol_no_final -> fila de 46 estados -> 6,73 KB (17x mais que o Backtracking)

5.4 Qualidade da Solução

Ambos os algoritmos retornam soluções exatas (ótimas). Quando existe subconjunto com soma igual a T, ambos o encontram. Quando não existe, ambos provam a impossibilidade.

A diferença está na solução específica retornada quando há múltiplas soluções válidas:

- O Backtracking retorna a primeira solução encontrada pelo percurso DFS.
- O B&B retorna a primeira solução encontrada pelo percurso BFS, tipicamente com menos elementos.

Exemplo no cenário fila_grande_bb.txt (T=30): Backtracking -> {2,3,4,5,6,10} | B&B -> {2,3,4,6,7,8}. Ambas igualmente válidas.

5.5 Eficiência de Poda

Cenário	Efic. Poda BT	Efic. Poda B&B	Vantagem
input (n=4)	23,5%	42,1%	B&B
padrao (n=8)	0,0%	0,0%	Empate
sol_imediata (n=10)	0,0%	0,0%	Empate
sem_solucacao (n=12)	50,0%	50,0%	Empate
poda_maxima (n=10)	39,1%	48,0%	B&B
sol_no_final (n=10)	36,9%	49,9%	B&B
fila_grande (n=15)	39,3%	5,6%	Backtracking

O B&B apresenta eficiência de poda superior em 4 de 7 cenários. O limite superior calculado via suffix_sum é mais preciso que a poda local do Backtracking. A exceção é o cenário fila_grande_bb, onde os muitos caminhos viáveis tornam a poda do B&B quase ineficaz (5,6%).

5.6 Escalabilidade

- O Backtracking manterá consumo de memória baixo e constante $O(n)$, degradando apenas em tempo.
- O B&B degradará simultaneamente em tempo e memória quando houver muitos estados viáveis, podendo tornar-se inviável para $n > 30$ em cenários adversos.

A vantagem do B&B em eficiência de poda compensa seu custo de memória apenas quando a poda é suficientemente agressiva para manter a fila pequena.

6. Justificativa das Técnicas de Otimização

6.1 Por que o Backtracking otimiza a busca?

O Backtracking reduz o espaço de busca de duas formas complementares:

Poda por excesso (elemento > remaining): quando um elemento candidato já ultrapassa o valor restante, não faz sentido incluí-lo. O ramo da inclusão é eliminado sem visitar nenhum de seus descendentes. Em conjuntos com elementos grandes em relação ao alvo, esta poda pode eliminar metade dos ramos em cada nível.

Poda por insuficiência (suffix_sum < remaining): quando a soma de todos os elementos restantes é menor que o valor ainda necessário, nenhuma combinação dos restantes atingirá o alvo. Este ramo inteiro - incluindo todos os seus filhos - é descartado.

Combinadas, essas podas transformam o espaço efetivo de busca de $O(2^n)$ para muito menos na prática: no cenário poda_maxima, apenas 23 nós são processados de um espaço teórico de $2^{10} = 1024$.

6.2 Por que o Branch and Bound otimiza a busca?

O B&B adiciona uma camada adicional de racionalidade: antes de expandir qualquer estado, calcula o melhor resultado possível a partir dele (limite superior). Se esse melhor resultado ainda for insuficiente para atingir o alvo, o estado é descartado completamente.

A chave da eficiência é o pre-computo das somas de sufixo ($O(n)$ uma única vez). Isso permite calcular o limite superior de qualquer estado em $O(1)$, sem re-somar os elementos restantes a cada verificação - tornando cada decisão de poda praticamente gratuita em custo computacional.

A exploração BFS garante que, quando uma solução é encontrada, ela é a primeira a ser alcançada naquele nível de profundidade, o que em alguns cenários significa encontrar soluções com menos elementos mais rapidamente.

7. Comparativo Final

Critério	Backtracking	Branch and Bound
Complexidade de tempo	$O(2^n)$	$O(2^n)$
Complexidade de espaço	$O(n)$	$O(2^n)$ pior caso
Uso de memória	Muito baixo e constante	Variável, pode ser alto

Velocidade - solucao precoce	Mais rapido (DFS chega logo)	Mais lento (BFS por nivel)
Velocidade - poda eficaz	Bom	Excelente (limite superior preciso)
Efic. de poda media	~28%	~34% (exceto fila_grande)
Caso favoravel	Solução nos primeiros elementos	Poda intensa elimina grandes blocos
Caso desfavoravel	Solução no final do array	Muitos caminhos viáveis simultaneos
Facilidade de implementação	Alta (recursao direta)	Media (fila + suffix_sum)
Qualidade da solução	Exata	Exata

8. Conclusao

Ambos os algoritmos implementados resolvem corretamente o Problema do Subconjunto Soma, sempre retornando soluções exatas. A diferença entre eles é essencialmente estrutural e estratégica, não de qualidade.

O Backtracking é a abordagem mais simples, com menor consumo de memória e melhor desempenho em cenários onde a solução está próxima do início do espaço de busca DFS. Sua memória $O(n)$ o torna viável para entradas maiores mesmo quando o tempo aumenta.

O Branch and Bound apresenta eficiência de poda superior na maioria dos cenários graças ao uso do limite superior calculado em $O(1)$ via somas de sufixo. No entanto, paga um custo de memória proporcional ao número de estados viáveis simultâneos, o que pode ser proibitivo em casos com muitas combinações viáveis.

A escolha entre as duas técnicas depende do perfil da instância:

- Use Backtracking quando a memória é limitada ou quando a solução tende a estar no início do espaço de busca.
- Use Branch and Bound quando o espaço de busca apresenta podas agressivas e a memória disponível é suficiente para sustentar a fila de estados.

Apêndice - Ambiente de Execução

Parametro	Valor
Linguagem	Python 3.11.4
Bibliotecas	Apenas padrao (time, tracemalloc, collections.deque)
Sistema Operacional	Windows 11
Medicao de tempo	time.perf_counter() - alta resolução
Medical de memoria	tracemalloc - pico de alocao em bytes, convertido para KB
Execucoes	Cada cenário rodado uma vez por algoritmo em ambiente isolado